Energy Based Models

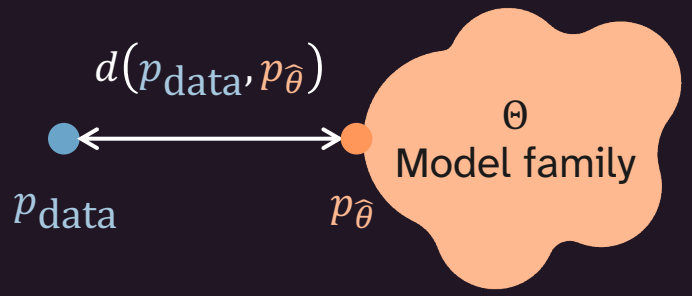
Chirag Raman
Assistant Professor
Department of Intelligent Systems, TUDelft

Generative Modeling DSAIT4030, 19/05/2026

Generative models: the story so far



$\mathbf{x}_i \sim p_{\text{data}}$
$i = 1,2,\ldots,N$



- define an objective $d(p_{\text{data}}, p_\theta)$ that captures 'closeness' between p_θ and p_{data}, and choose the model closest to the truth

$$\hat{\theta} \in \arg\min_\theta d(p_{\text{data}}, p_\theta)$$

Cat images from RedBubble

Generative models: the story so far

All the models we have considered are composed of probability densities!

Latent Variable Models $p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$

Flow-Based Models $p(\mathbf{x}) = p(\mathbf{z}) \left|\det \left(\mathbf{J}_f(\mathbf{z})\right)\right|^{-1}, \quad \mathbf{z} = f^{-1}(\mathbf{x})$

Autoregressive Models $p(\mathbf{x}) = \prod_{d=1}^{D} p(x_d \mid \mathbf{x}_{<d})$

Parameterizing probability densities

Requirements of a pdf

1. Non-negativity: $p(\mathbf{x}) \geq 0 \quad \text{for all } \mathbf{x} \in \mathbb{R}^D$
2. Normalization: $\int_{\mathbb{R}^D} p(\mathbf{x})\, d\mathbf{x} = 1$

... and measurability, integrability, smoothness

Choosing non-negative functions is easy ...

$$p(\mathbf{x}) = |f_\theta(\mathbf{x})|, \quad p(\mathbf{x}) = f_\theta(\mathbf{x})^2, \quad p(\mathbf{x}) = \exp(f_\theta(\mathbf{x}))$$

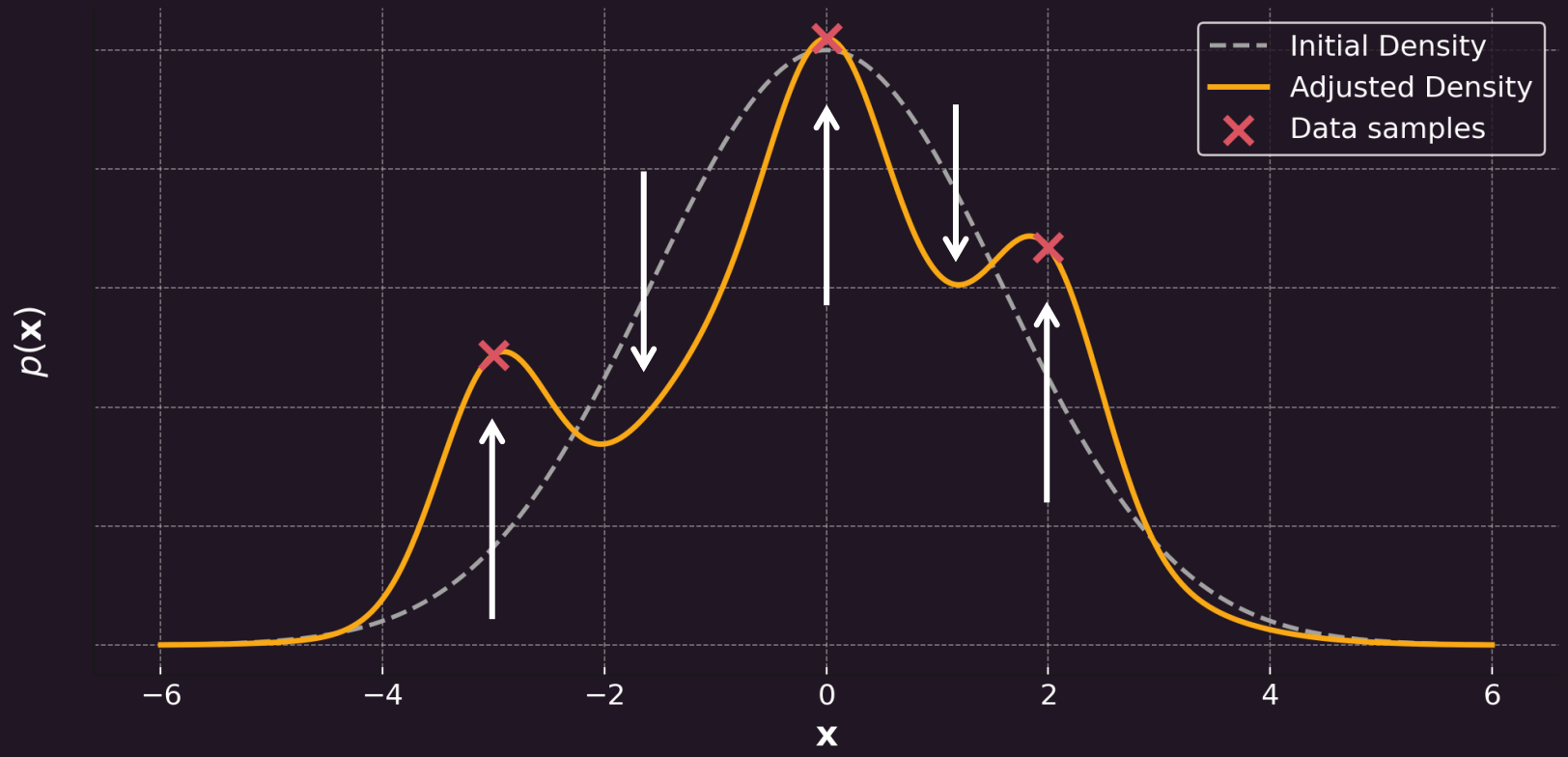
where f_θ can be some neural network

but the functions might not be normalized!

Parameterizing probability densities

Having a fixed volume is key!

When learning from data, we want to make $\mathbf{x}_{\text{data}}$ more likely, so we want to increase $p(\mathbf{x}_{\text{data}})$ but decrease density elsewhere





Normalizing a learnable function

Problem: Finding a non-negative function is easy, but a non-negative and normalized one is hard

💡 Explicitly normalize a non-negative function by dividing by its volume:

$$p_\theta(\mathbf{x}) = \frac{f_\theta(\mathbf{x})}{\int f_\theta(\mathbf{x}')\, d\mathbf{x}'}$$

1. Proof of non-negativity

$$\text{Let } Z = \int f_\theta(\mathbf{x}')\, d\mathbf{x}' \quad \text{and suppose } Z > 0.$$

$$\text{Since } f_\theta(\mathbf{x}) \geq 0, \quad \text{it follows that } p_\theta(\mathbf{x}) = \frac{f_\theta(\mathbf{x})}{Z} \geq 0.$$

Normalizing a learnable function

Problem: Finding a non-negative function is easy, but a non-negative and normalized one is hard

💡 Explicitly normalize a non-negative function by dividing by its volume:

$$p_\theta(\mathbf{x}) = \frac{f_\theta(\mathbf{x})}{\int f_\theta(\mathbf{x}')\, d\mathbf{x}'}$$

2. Proof of normalization

$$\int p_\theta(\mathbf{x})\, d\mathbf{x} = \int \frac{f_\theta(\mathbf{x})}{\int f_\theta(\mathbf{x}')\, d\mathbf{x}'}\, d\mathbf{x} = \frac{\int f_\theta(\mathbf{x})\, d\mathbf{x}}{\int f_\theta(\mathbf{x}')\, d\mathbf{x}'} = 1$$

Designing models with tractable volume

So far, we've ensured normalization by making smart choices — either in the density form with analytical volume or the model structure:

- Gaussian

$$p_{\mu,\sigma}(x) = \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right) \quad \Rightarrow \quad \int p_{\mu,\sigma}(x)\,dx = \sigma\sqrt{2\pi}$$

- Laplace

$$p_\lambda(x) = \exp(-\lambda|x|) \quad \Rightarrow \quad \int p_\lambda(x)\,dx = \frac{2}{\lambda}$$

Designing models with tractable volume

So far, we've ensured normalization by making smart choices — either in the density form with analytical volume or the model structure:

- Latent Variable Models

$$\int p(\mathbf{x})\, d\mathbf{x} = \int \left(\int p(\mathbf{x} \mid \mathbf{z})\, d\mathbf{x} \right) p(\mathbf{z})\, d\mathbf{z} = \int p(\mathbf{z})\, d\mathbf{z} = 1$$

- Flow-Based Models

  f is bijective, we track volume change through the change of variables

- Autoregressive Models

$$\int p(\mathbf{x})\, d\mathbf{x} = \int \cdots \int \prod_{i=1}^{D} p(x_i \mid \mathbf{x}_{<i})\, dx_D \cdots dx_1 = 1$$

Designing models with tractable volume

So far, we've ensured normalization by making smart choices — either in the density form with analytical volume or the model structure.

But this can limit the expressiveness of the model.

Can we design models where we don't have an analytical volume?

Energy Based Models (EBMs)

EBMs specify a probability density of $\mathbf{x} \in \mathbb{R}^D$ as a Gibbs distribution

$$p_\theta(\mathbf{x}) = \frac{\exp\left(-E_\theta(\mathbf{x})\right)}{Z_\theta} \quad \text{where} \quad Z_\theta = \int \exp\left(-E_\theta(\mathbf{x}')\right) d\mathbf{x}'$$

where

$E_\theta : \mathbb{R}^D \to \mathbb{R}$, the energy function, maps data points to scalar energy

$Z_\theta : \Theta \to \mathbb{R}^+$, the normalizing constant, known as the partition function

Crucially, the energy function can be any function that returns a non-negative scalar; It does not need to integrate to 1!

Why ‘energy’ and ‘partition’?

Physics inspiration originates from Boltzmann, defining the probability of a system being in state $\mathbf{x}$ with energy $E(\mathbf{x})$ at temperature T is

$$p(\mathbf{x}) = \frac{1}{Z} \exp\left(-\frac{E(\mathbf{x})}{kT}\right)$$

In Machine Learning, we set $kT = 1$, yielding the form of the EBM.

$E_\theta(\mathbf{x})$ behaves like physical *energy* : low energy ⇒ high likelihood

Z_θ *‘partitions’ or divides* the probability mass across all states

EBMs borrow their form from Gibbs and intuition from Boltzmann—
using energy to shape a probability landscape

Why use the exponential?

We only need a nonnegative function to define a pdf of the form

$$p_\theta(\mathbf{x}) = \frac{f_\theta(\mathbf{x})}{\int f_\theta(\mathbf{x}')\, d\mathbf{x}'}$$

so why choose $f_\theta(\mathbf{x}) = \exp\left(-E_\theta(\mathbf{x})\right)$?

- Allows for large variations in probability (small changes in $E_\theta(\mathbf{x})$ lead to exponential changes in $p_\theta(\mathbf{x})$, without needing $E_\theta(\mathbf{x})$ to be non-smooth)
- Log-probability becomes linear in energy -> easier to optimize
- Common distributions (exponential families) can be written in this form
- Arise under fairly general assumptions in statistical physics (e.g. maximum entropy, thermodynamics)

Properties of EBMs

Extreme flexibility and expressivity

- $E_\theta(\mathbf{x})$ can be any integrable function, e.g. neural network
- No constraints on normalization structure (unlike flows or autoregressive models)

Computing Z_θ numerically is hard

- Sampling from $p_\theta(\mathbf{x})$ is hard
- Evaluating and optimizing $p_\theta(\mathbf{x})$ (learning) is hard

No representation learning (although we can add latent variables, e.g Restricted Boltzmann machines)

Why is training EBMs hard?

Maximum likelihood training involves $\log p_\theta(\mathbf{x})$ but the partition function Z_θ is intractable. Note that

$$\log p_\theta(\mathbf{x}) = \log \left(\frac{\exp(-E_\theta(\mathbf{x}))}{Z_\theta} \right) = -E_\theta(\mathbf{x}) - \log Z_\theta$$

Given a dataset of $\mathbf{X}$ with samples $\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$, the MLE objective is

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} [-E_\theta(\mathbf{x}_i) - \log Z_\theta] = -\frac{1}{N} \sum_{i=1}^{N} E_\theta(\mathbf{x}_i) - \log Z_\theta$$

- First term: data dependent — pushes energy down at observations
- Second term: global — depends on all possible samples, intractable

Training EBMs: Gradient of the MLE Objective

$$
\begin{aligned}
\nabla_\theta \log p_\theta(\mathbf{x}) &= \nabla_\theta \left[-E_\theta(\mathbf{x}) - \log Z_\theta \right] \\
&= -\nabla_\theta E_\theta(\mathbf{x}) - \nabla_\theta \log Z_\theta \\
&= -\nabla_\theta E_\theta(\mathbf{x}) - \frac{1}{Z_\theta} \nabla_\theta Z_\theta && \text{... Chain rule} \\
&= -\nabla_\theta E_\theta(\mathbf{x}) - \frac{1}{Z_\theta} \nabla_\theta \int \exp(-E_\theta(\mathbf{x}')) d\mathbf{x}' \\
&= -\nabla_\theta E_\theta(\mathbf{x}) + \frac{1}{Z_\theta} \int \exp(-E_\theta(\mathbf{x}')) \nabla_\theta E_\theta(\mathbf{x}') \, d\mathbf{x}' && \text{... Chain rule} \\
&= -\nabla_\theta E_\theta(\mathbf{x}) + \int \frac{\exp(-E_\theta(\mathbf{x}'))}{Z_\theta} \nabla_\theta E_\theta(\mathbf{x}') \, d\mathbf{x}' \\
&= -\nabla_\theta E_\theta(\mathbf{x}) + \int p_\theta(\mathbf{x}') \nabla_\theta E_\theta(\mathbf{x}') \, d\mathbf{x}' = \boxed{-\nabla_\theta E_\theta(\mathbf{x}) + \mathbb{E}_{\mathbf{x}' \sim p_\theta} \left[\nabla_\theta E_\theta(\mathbf{x}') \right]}
\end{aligned}
$$

Interpreting the Gradient: Push-Pull Dynamics

$$\nabla_\theta \log p_\theta(\mathbf{x}) = -\nabla_\theta E_\theta(\mathbf{x}) + \mathbb{E}_{\mathbf{x}' \sim p_\theta}\left[\nabla_\theta E_\theta(\mathbf{x}')\right]$$

First term pushes energy down at observed data -> samples are more likely

Second term pulls energy up on model samples -> suppresses non-data regions

The push-pull dynamic creates energy wells around the data samples

Let's look at the second term

Positive : gradient inside expectation, no reparameterization needed!

Negative : requires sampling from $p_\theta(\mathbf{x})$ to estimate the expectation!

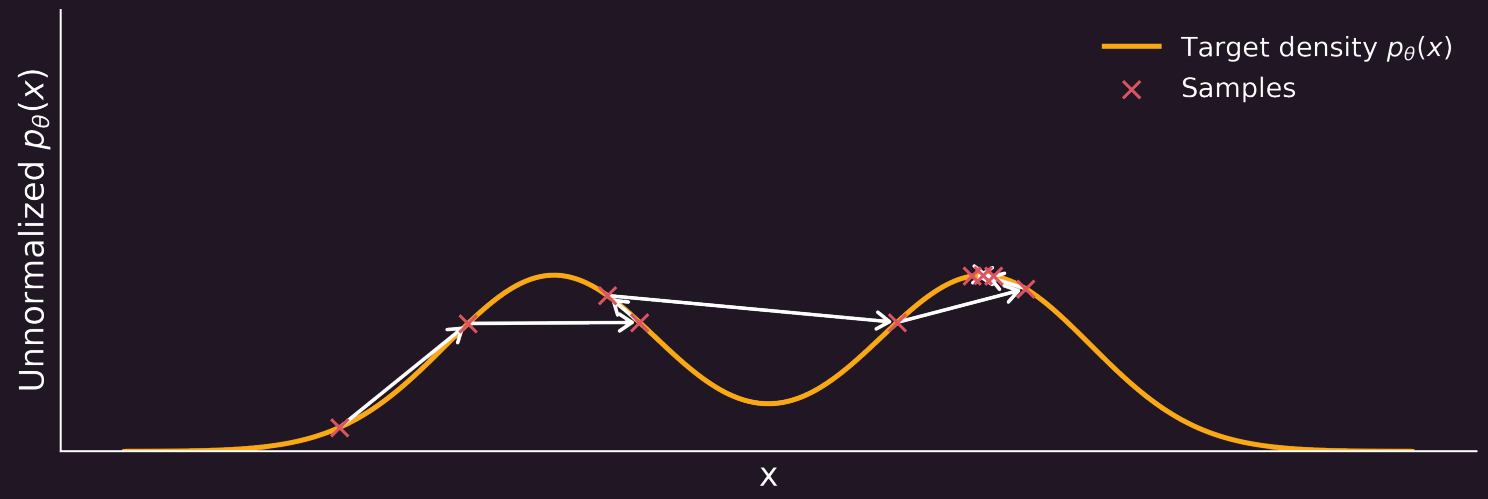
Approximating the gradient by sampling

To estimate $\mathbb{E}_{\mathbf{x}' \sim p_\theta}\left[\nabla_\theta E_\theta(\mathbf{x}')\right]$ we need samples from $p_\theta(\mathbf{x}) \propto \exp(-E_\theta(\mathbf{x}))$

💡 Use *Markov Chain Monte Carlo (MCMC)*: simulate a Markov chain whose stationary distribution is $p_\theta(\mathbf{x})$

$$\mathbf{x}^{(0)} \to \mathbf{x}^{(1)} \to \cdots \to \mathbf{x}^{(t)} \sim p_\theta(\mathbf{x})$$

$$\left(\text{Markov assumption: } p(\mathbf{x}^{(i+1)} \mid \mathbf{x}^{(0)}, \ldots, \mathbf{x}^{(i)}) = p(\mathbf{x}^{(i+1)} \mid \mathbf{x}^{(i)})\right.$$



! Samples are not iid

! Eventually, samples approximate the stationary distribution

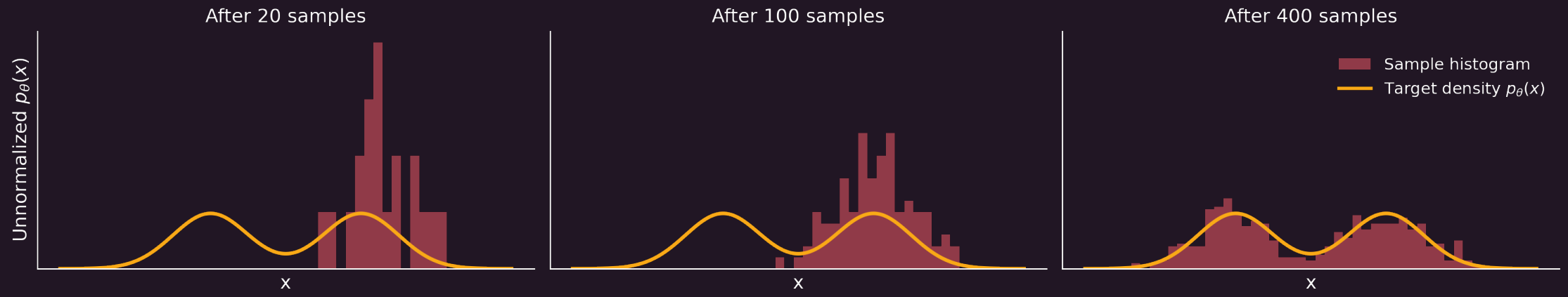
Approximating the gradient by sampling

To estimate $\mathbb{E}_{\mathbf{x}' \sim p_\theta}\left[\nabla_\theta E_\theta(\mathbf{x}')\right]$ we need samples from $p_\theta(\mathbf{x}) \propto \exp(-E_\theta(\mathbf{x}))$

💡 Use *Markov Chain Monte Carlo (MCMC)*: simulate a Markov chain whose stationary distribution is $p_\theta(\mathbf{x})$

$$\mathbf{x}^{(0)} \to \mathbf{x}^{(1)} \to \cdots \to \mathbf{x}^{(t)} \sim p_\theta(\mathbf{x})$$

$$\left(\text{Markov assumption: } p(\mathbf{x}^{(i+1)} \mid \mathbf{x}^{(0)}, \ldots, \mathbf{x}^{(i)}) = p(\mathbf{x}^{(i+1)} \mid \mathbf{x}^{(i)})\right.$$



Metropolis-Hastings: Building the Markov chain

We use a proposal distribution $q(\mathbf{x}' \mid \mathbf{x}^{(t)})$ to explore the space.

Starting with an initial sample $\mathbf{x}^{(0)}$,

1. **Propose:** $\mathbf{x}' \sim q(\mathbf{x}' \mid \mathbf{x}^{(t)})$
2. **Compute:** $\alpha = \min\left(1, \frac{p_\theta(\mathbf{x}')\, q(\mathbf{x}^{(t)} \mid \mathbf{x}')}{p_\theta(\mathbf{x}^{(t)})\, q(\mathbf{x}' \mid \mathbf{x}^{(t)})}\right)$
3. **Accept/reject:** $\mathbf{x}^{(t+1)} = \begin{cases} \mathbf{x}', & \text{with probability } \alpha \\ \mathbf{x}^{(t)}, & \text{otherwise} \end{cases}$

! Accept/reject ensures that the chain converges to the target distribution $p_\theta(\mathbf{x})$

! Acceptance probability only involves ratios of $p_\theta(\mathbf{x})$ -> partition function cancels

Langevin dynamics: smarter MCMC with gradients

We can use the gradient of the log density (Stein score) to guide sampling!

- Instead of proposing blindly, move down the energy using the gradient

$$\nabla_{\mathbf{x}} \log p_\theta(\mathbf{x}) = -\nabla_{\mathbf{x}} E_\theta(\mathbf{x}) - \underbrace{\nabla_{\mathbf{x}} \log Z_\theta}_{\text{does not depend on } \mathbf{x}} = -\nabla_{\mathbf{x}} E_\theta(\mathbf{x})$$

- Langevin dynamics adds noise to avoid getting stuck:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \frac{\eta}{2} \nabla_{\mathbf{x}} E_\theta(\mathbf{x}_t) + \sqrt{\eta}\, \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Note: This defines a proposal distribution $q(\mathbf{x}' \mid \mathbf{x}) = \mathcal{N}\left(\mathbf{x}' \mid \mathbf{x} - \frac{\eta}{2} \nabla_{\mathbf{x}} E_\theta(\mathbf{x}),\, \eta \mathbf{I}\right)$